MANIT KHERA

One cardboard robot, three university units, and a student who could not leave it alone

0412 761 206 | manitkhera26@gmail.com | github.com/uiqvb | linkedin.com/in/manit-khera-603957252

About me

Hi, I'm Manit. Third-year Software Engineering student at Deakin, applying for Syncrowin's Early Career Data Scientist role, and apparently incapable of leaving a finished project alone.

Here is the pattern. I get an assignment. I finish it. Then I look at it and think: what else could this do? Last trimester that meant joining two completely unrelated assignments, a robot from one unit and a backend from another, purely because I wanted to watch my own code push a real thing across the floor. A month of build, break, rebuild later, the backend spoke and the robot listened.

I should credit my unit chair, Maksym. Every time I pitched an idea he would say "nah, you can't do that". Not to put me off: he had worked out that I do my best work trying to prove him wrong. Then, when I got there, he moved the target further out. He rage baited me through the whole unit, and in between we would sit there like two kids going "okay, but what if...". It was a proper love-hate arrangement and I would do it again.

Then the robot went to Open Day. Then into a third unit, where I asked what happens if there are 5,000 of it. So this is not really three projects. It is one robot that I keep volunteering for jobs it never applied for.

You asked about a data science project I'm proud of. I haven't trained a model yet, so I won't pretend any of this is one. The closest is Chapter 1: a robot whose own sensor logs told me what was wrong with it, after I had spent a lot of money guessing.

Meet the robot

Build: two wheels, six sensors, one cardboard body.

Jobs so far: laundry attendant, class demo, Open Day exhibit, and the model for 5,000 imaginary colleagues.

Good at: running for shelter when it rains, and retracing its steps when it gets lost.

Bad at: wonky tape, and any battery below 90 percent.

Marks it earned me: 93 and 100/100.

it has never once complained about the extra work

What your Early Career Data Scientist does, and where I have done something close

Sensor data

Six sensors on the robot, read by an Arduino and sent every five seconds over MQTT to Python I wrote on a Raspberry Pi. Chapter 1.

Finding what's wrong

Tracked an intermittent power fault down in the robot's own logs, not with a multimeter. Chapter 1.

Experiments

Wet tests, fake sensor readings, and 80 benchmark runs with three repeats per setup and the median reported. Chapter 3.

SQL, Python, Azure

PostgreSQL behind both backends, Python on the Pi, and a 5,000-robot load test on Azure. Chapters 1 to 3.

Explaining results

A written report on the scaling work, and a live demo for students and parents at Deakin Open Day. Chapters 2 and 3.

Chapter 1. The robot that saves your clothes

Smart Clothesline Robot. Embedded Systems unit, awarded 93.

Arduino Nano, C++, Raspberry Pi, Python/Flask, MQTT

Follow the build: 1 First demo → 2 Second demo → 3 Build write-up on Hackster.io → 4 Source code on GitHub

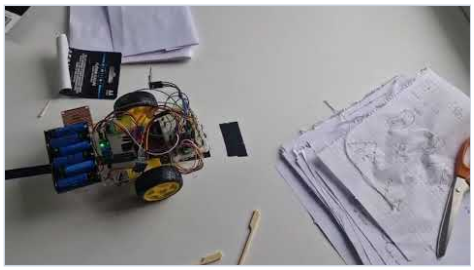
The idea. Clothes get rained on. So a small robot carries them between a sunny spot and a sheltered one, follows a line of black tape to get there, and turns to face the sun while it waits. I built and wired it myself: two wheels, three line sensors, two light sensors, one rain sensor and a cardboard body.

How I tested it. I got it wet. On purpose. Wetting the rain sensor sent it off to shelter, as it should. Then I lied to it: I fed it fake rain readings to check it would trust the weather forecast over a broken sensor. It did.

The real problem. The robot had a voltage problem I could not see. On fresh batteries it behaved perfectly. Once they dropped below about 90 percent, it started acting up. So I kept buying new ones, probably $100 worth, before I went back through the robot's logs. The boot ID and reset cause in each message showed it was browning out, and that led me to a faulty voltage converter.

I now own a large pile of batteries that are 90% full and useless to me

What I took from it. I should have checked the logs first. I now put fields like that into everything I build, because they are what made the fault findable.



The robot on its tape track, from the first demo.

yes, the body really is cardboard

6 sensors, all wired by hand

93 unit mark

Chapter 2. Giving it a brain it never asked for

Robot Controller API. Backend Development unit, awarded 100/100.

.NET 8, C#, PostgreSQL, Arduino C++, Docker, Jenkins

Follow the build: 1 First demo, in class with my unit chair → 2 Open Day demo →

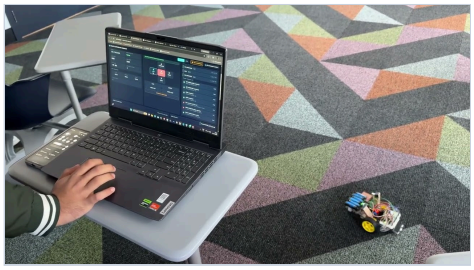
3 GitHub: first build, Open Day branch, deploying branch

The idea. A different unit entirely. I built a backend that does all the thinking, so a robot only has to ask "what next?" over Wi-Fi and do as it is told. Then I pointed it at the clothesline robot to see what would happen. For a while, not much: sometimes the backend worked and the robot ignored it, and sometimes the robot moved while the software believed something else entirely. So every command and result now goes into a PostgreSQL history I can query.

My favourite what-if. What if the Wi-Fi drops? A robot with no plan of its own would be lost. So I built for it before it ever happened: every command arrives with its own undo. After 7 seconds of silence the robot retraces its last five moves by itself, back toward signal, then tells the server what it did. The server checks that story in a simulation before believing it. When I tested it on the real robot, it ended up exactly where the server predicted.

Party trick. It is also an RC car. You can drive it from the dashboard with the WASD keys, like a video game. There is nothing fancy underneath: no special radio link, only ordinary web requests. The robot keeps asking the server "anything for me?" and the server answers.

Where it went. A first demo in class with my unit chair, then selected for Deakin Open Day, where I ran it live for prospective students and their parents.



My dashboard driving the robot across the floor.

the robot is the small thing, bottom right

7 s	100/100
of silence before it rescues itself	unit mark

Chapter 3. What if there were 5,000 of them?

Scaling the Robot Dispatch API. Distributed Programming unit.

C#, ASP.NET Core, PostgreSQL, Azure, Docker, NBomber, xUnit

Follow the build: 1 Starting point: the Chapter 2 code → 2 Rebuilt code on GitHub → 3 Demo video → 4 Full report

The idea. One robot worked. So in a third unit I asked the uncomfortable question, built a pretend fleet in the cloud, and pointed it at my own backend. At 5,000 robots it completed zero jobs.

What I did about it. I went looking and found 12 problems, including one where two robots could grab the same job. I fixed them, taught the server to say "try again in a second" instead of falling over, added a second server, and ran it all again.

How I know it worked. 80 benchmark runs, each grid setup repeated three times with the median reported, plus 54 automated tests. Median claim time at 400 robots went from 766.5 ms to 59.5 ms.

Still unknown. The new breaking point. My pretend fleet gave up before the backend did, which I am choosing to take as a compliment.

Jobs finished per second, 400 robots

Before	177
After, 1 server	608 (3.44×)
After, 2 servers	1,069 (6.05×)

Jobs finished per second, 5,000 robots

Before	0
After, 2 servers	~1,008

from nothing at all to a thousand a second

The data science side, honestly

What I haven't done. Statistics, probability, linear algebra and machine learning. No forecasting models and no PyTorch yet.

What I have done. Collected sensor data, checked whether it could be trusted, and measured systems properly before believing a number.

What I want to learn. The modelling side, from your AI engineers, on data from real sites instead of a cardboard robot.

What I am looking for. The Early Career Data Scientist role at Syncrowin; I am available immediately. I am strongest where software meets real hardware and the data it produces, and I want to learn to do far more with that data than I can today. The habit on this page comes with me: try it, break it on purpose, measure it, write it down, and bring it back for another round.